# PRO WRESTLING ROSTER API

PREPARED FOR **Callen**

| Stack — | Due — | Time — | Endpoints — |
|---|---|---|---|
| C# · ASP.NET Core · EF Core · SQLite | First class session after break | 10–14 hours across the week | 6 (5 CRUD + 1) |

THE BRIEF

An independent wrestling promotion books shows every weekend and keeps its roster in a spreadsheet nobody trusts. The booker wants an API to manage the talent: who is on the roster, what they weigh in at, their win-loss record, whether they are a fan favorite or a villain, and whether they are cleared to compete. Every talent is a **Wrestler**.

/01

## WHAT YOU ARE BUILDING

Everything in this handout is required unless it is in the Stretch section.

Read the whole document once before writing any code.

| | |
|---|---|
| PROJECT | `WrestlingRoster.Api`, an ASP.NET Core Web API using controllers |
| DATABASE | SQLite file `wrestlingroster.db` managed by EF Core migrations |
| TABLE | One table, `Wrestlers`, holding `Wrestler` rows. No relationships to other tables. |
| LAYERS | `WrestlersController` → `IWrestlerService` → `IWrestlerRepository` → `AppDbContext` |
| REPOSITORY | Exactly five methods: `GetAll`, `GetById`, `Create`, `Update`, `Delete` |
| ENDPOINTS | Six. Full CRUD on `/api/wrestlers` plus one domain endpoint, `PATCH /api/wrestlers/{id}/turn` |

RULES — Four business rules enforced in the service layer, returning 400 or 409

### What this practices

- Wiring the full n-tier chain for one resource and registering every piece with dependency injection.
- Keeping each layer honest: HTTP in the controller, decisions in the service, EF Core in the repository.
- Designing DTOs that differ from the entity on purpose: hidden server-set fields and one computed field.
- Choosing the right status code, especially the line between 400 and 409.
- Writing filtering and rule checks with LINQ in the service, since the repository only hands back the whole table or one row.

/02

# REQUIRED ARCHITECTURE

One table, one controller, one service, one repository.

If you add a second table in the stretch goals, it gets its own copy of all four, following the same pattern.

| 01 | → 02 | → 03 | → 04 | → 05 |
|---|---|---|---|---|
| **Controller** | **Service** | **Repository** | **DbContext** | **SQLite** |
| Routes, model binding, status codes. Calls the service interface. Knows nothing about EF Core. | Business rules, validation beyond attributes, mapping entity ↔ DTO, LINQ filtering. Calls the repository interface. | The only place EF Core appears. Five methods, returns entities, no DTOs, no HTTP. | `DbSet<Wrestler>` and the SQLite connection. Nothing else. | The `wrestlingroster.db` file created by migrations. |

### Folder layout

```
WrestlingRoster.Api/
├── Controllers/
│   └── WrestlersController.cs
├── Services/
│   ├── IWrestlerService.cs
│   └── WrestlerService.cs
├── Repositories/
│   ├── IWrestlerRepository.cs
│   └── WrestlerRepository.cs
├── Models/
│   └── Wrestler.cs
├── DTOs/
│   ├── CreateWrestlerDto.cs
│   ├── UpdateWrestlerDto.cs
│   └── WrestlerResponseDto.cs
├── Data/
│   └── AppDbContext.cs
├── Migrations/                    generated
├── Program.cs
```

### The repository contract

REPOSITORIES/IWRESTLERREPOSITORY.CS

```
public interface IWrestlerRepository
{
    IEnumerable<Wrestler> GetAll();
    Wrestler? GetById(int id);
    Wrestler Create(Wrestler wrestler);
    Wrestler Update(Wrestler wrestler);
    bool Delete(int id);
}
```

Every repository you write this term has exactly this shape. `GetAll` is `ToList()` over the whole table. No `async` or `await` anywhere in this project; we have not covered it yet. There are no search or filter methods here on purpose: that work belongs to the service, using LINQ over what `GetAll` returns.

```
├── appsettings.json
└── README.md
```

If we put interfaces in an `Interfaces/` folder in class, do that instead. The point is that every piece exists and is named consistently.

### Layer rules — REQUIRED

| | |
|---|---|
| 01 | The controller depends only on `IWrestlerService`. It never touches the repository or `AppDbContext`. |
| 02 | The service depends only on `IWrestlerRepository`. It never returns an entity to the controller, only DTOs. |
| 03 | The repository is the only class that uses `AppDbContext` or any `Microsoft.EntityFrameworkCore` type. |
| 04 | All three are registered in `Program.cs` with `AddScoped`, interface to implementation. |
| 05 | `GetById` returns `null` and `Delete` returns `false` when the id does not exist. The service turns that into a 404; the repository never throws for a missing row. |

/03

## DATA MODEL

`string?` means nullable. Everything else is required.

Create `Models/Wrestler.cs` with these properties. Use the C# types shown. Simple limits can be enforced with validation attributes on the DTOs; the service handles anything that needs a comparison or a lookup.

| PROPERTY | TYPE | RULES |
|---|---|---|
| `Id` | `int` | Primary key, generated by the database |
| `RingName` | `string` | Required, max 60, unique |
| `RealName` | `string` | Required, max 100 |
| `Brand` | `string` | Required, max 50 (the show they appear on, e.g. Friday Fury) |
| `WeightClass` | `string` | One of `Cruiserweight`, `Heavyweight`, `Super Heavyweight` |
| `Alignment` | `string` | Exactly `Face` (hero) or `Heel` (villain) |
| `Finisher` | `string` | Required, max 60 (their finishing move) |
| `Wins` | `int` | 0 or more |
| `Losses` | `int` | 0 or more |
| `DebutYear` | `int` | 1950 through 2100 |

| PROPERTY | TYPE | RULES |
|---|---|---|
| Status | `string` | One of `Active`, `Injured`, `Retired` |

There are no foreign keys or navigation properties in this assignment. The table stands alone.

/04

## DTOS

Validation attributes go on the DTOs, not the entity.

A failed attribute becomes an automatic 400.

Clients never see or send the `Wrestler` entity. Three DTO classes in `DTOs/` define the API's shape.

| | |
|---|---|
| `CreateWrestlerDto` | All fields except `Id`. Put the `[Required]`, `[MaxLength]`, and `[Range]` attributes here so the simple checks return 400 automatically. |
| `UpdateWrestlerDto` | Same shape as the create DTO. A `PUT` replaces every editable field. |
| `WrestlerResponseDto` | All fields plus a computed `Record` (`string`) such as `"12-3"` built from `Wins` and `Losses`. |

COMPUTED

`Record` does not exist on the entity or in the database. The service calculates it every time it maps an entity to a response DTO. That mapping should live in one private method in the service so every endpoint returns the same shape.

/05

## ENDPOINTS

The sixth endpoint is the one that makes this API *this* API. Build CRUD first, then this.

Six endpoints, all on `WrestlersController` with `[ApiController]` and `[Route("api/wrestlers")]`. Not every endpoint can produce every error; the error column lists the ones this assignment's rules can trigger.

| METHOD | ROUTE | BODY | SUCCESS | ERRORS |
|---|---|---|---|---|
| **GET** | `/api/wrestlers` | none | 200 list of response DTOs | — |
| **GET** | `/api/wrestlers/{id}` | none | 200 response DTO | 404 |
| **POST** | `/api/wrestlers` | `CreateWrestlerDto` | 201 + Location header | 400 · 409 |
| **PUT** | `/api/wrestlers/{id}` | `UpdateWrestlerDto` | 204 | 400 · 404 · 409 |
| **DELETE** | `/api/wrestlers/{id}` | none | 204 | 404 · 409 |
| **PATCH** | `/api/wrestlers/{id}/turn` | none | 200 with the updated wrestler | 404 unknown id · 409 when `Status` is not `Active` |

| | |
|---|---|
| THE SIXTH | The heel turn (or face turn). Flip `Alignment`: `Face` becomes `Heel`, `Heel` becomes `Face`. Load the wrestler, flip the value, call `Update`, return the response DTO. An injured or retired wrestler is not on TV, so they cannot turn. |

### Status code conventions

| CODE | USE IT WHEN |
|---|---|
| `200 OK` | Successful `GET`, and the sixth endpoint when it returns the changed record |
| `201 Created` | Successful `POST`. Return `CreatedAtAction(nameof(GetById), new { id }, dto)` so the `Location` header is set |
| `204 No Content` | Successful `PUT` and `DELETE`. No body |
| `400 Bad Request` | Validation failed: a required field is missing, a number is out of range, a value is not one of the allowed words |
| `404 Not Found` | The id (or lookup value) does not exist |
| `409 Conflict` | The request was well-formed but a business rule says no: duplicates, a state that cannot change, a limit that is already reached |

/06

## BUSINESS RULES

**400** the request itself is wrong.

**409** the request is fine but the current state says no.

These live in `WrestlerService`. The number on the right is the status code the client must receive when the rule is broken. The controller's job is only to translate the service's answer into that code.

| | | |
|---|---|---|
| 01 | `RingName` must be unique, compared case-insensitively. There is only one `"The Night Owl"` on this roster. | **409** |
| 02 | `Wins` and `Losses` must be 0 or more and `DebutYear` must be 1950 through 2100. Use validation attributes on the DTOs. | **400** |
| 03 | `WeightClass`, `Alignment`, and `Status` must each be one of their allowed values. Check these in the service and reject anything else, including different casing. | **400** |
| 04 | Only an `Active` wrestler can turn. Calling the turn endpoint on an `Injured` or `Retired` wrestler is a conflict. | **409** |

| | |
|---|---|
| PATTERN | Because the repository only offers `GetAll` and `GetById`, every uniqueness check, count, and filter is LINQ in the service: `var all = _repo.GetAll();` then `all.Any(...)`, `all.Count(...)`, `all.Where(...)`. The list is already in memory, so `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` works for case-insensitive comparisons. |

/07

# STRETCH GOALS

Stretch work only counts if the README says it exists.

Optional. Only start these once all six endpoints and all four rules work, and note in the README which ones you attempted.

### Second table: Championship — STRETCH A

Add a `Championship` model (`Title` unique, `Division`, `CurrentHolder` as plain text, `LastDefendedOn` date) with its own repository, service, controller, and the same five CRUD endpoints. No relationship to `Wrestler`; it is a separate table. Add `PATCH /api/championships/{id}/defend` that sets `LastDefendedOn` to today.

### Record a match — STRETCH B

`PATCH /api/wrestlers/{id}/result` with a body of `{ "result": "Win" }` or `"Loss"` that increments the right counter, returning 400 for any other word and 409 if the wrestler is not `Active`.

### Search — STRETCH C

`GET /api/wrestlers/search?brand=&alignment=&weightClass=` with every parameter optional and case-insensitive.